

UML, herramientas y conceptos básicos de la POO

UML (Unified Modeling Language) es el estándar para diseñar y documentar sistemas de software orientados a objetos. Se creó en 1997 y define un conjunto de diagramas gráficos para visualizar la estructura y el comportamiento de una aplicación antes de programarla.

Dos grandes familias de diagramas UML 2.5

Estructura (qué piezas tiene)

- Clases (este tema)
- Objetos
- Componentes
- Despliegue
- Paquetes

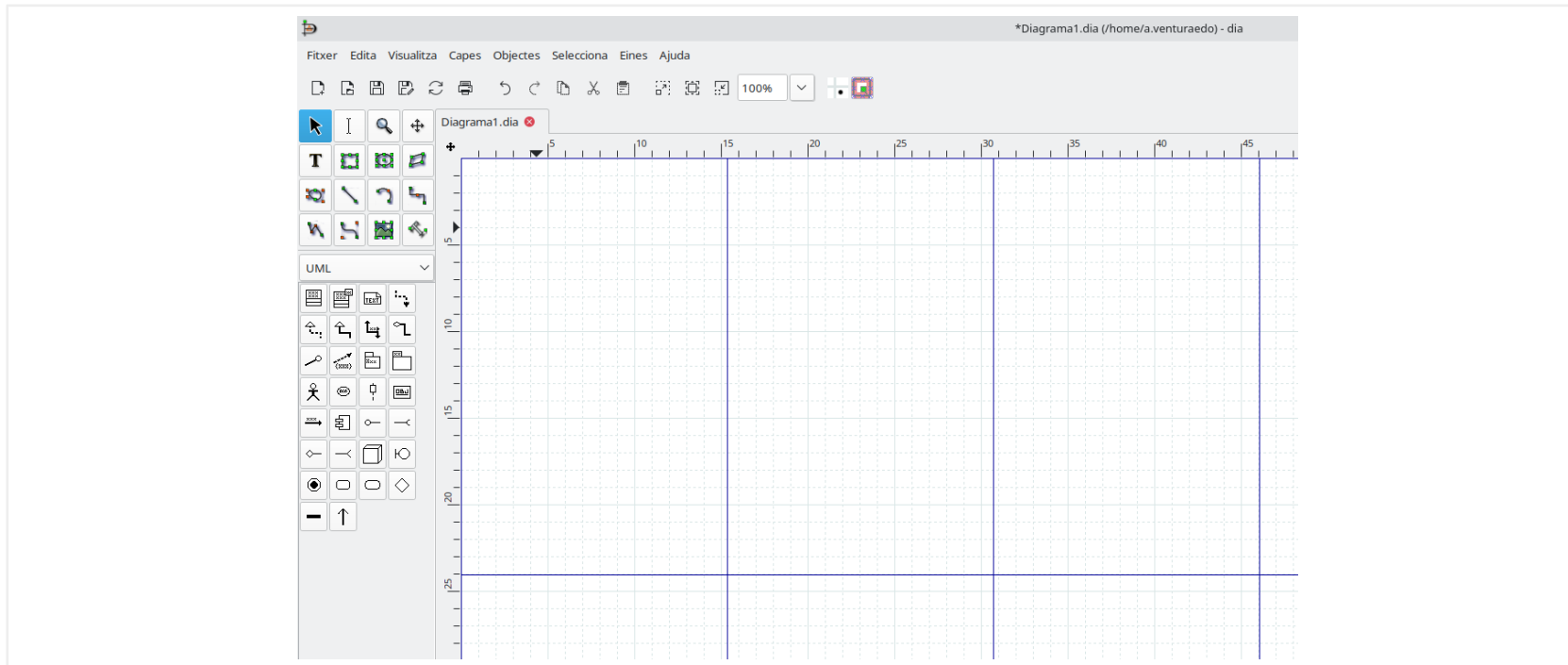
Comportamiento (qué hace)

- Casos de uso
- Secuencia
- Comunicación
- Actividades
- Estados

UML es una herramienta de comunicación, no un fin en sí mismo. En metodologías ágiles se usa de forma ligera; en proyectos más formales se documenta más.

DIA: herramienta principal del curso

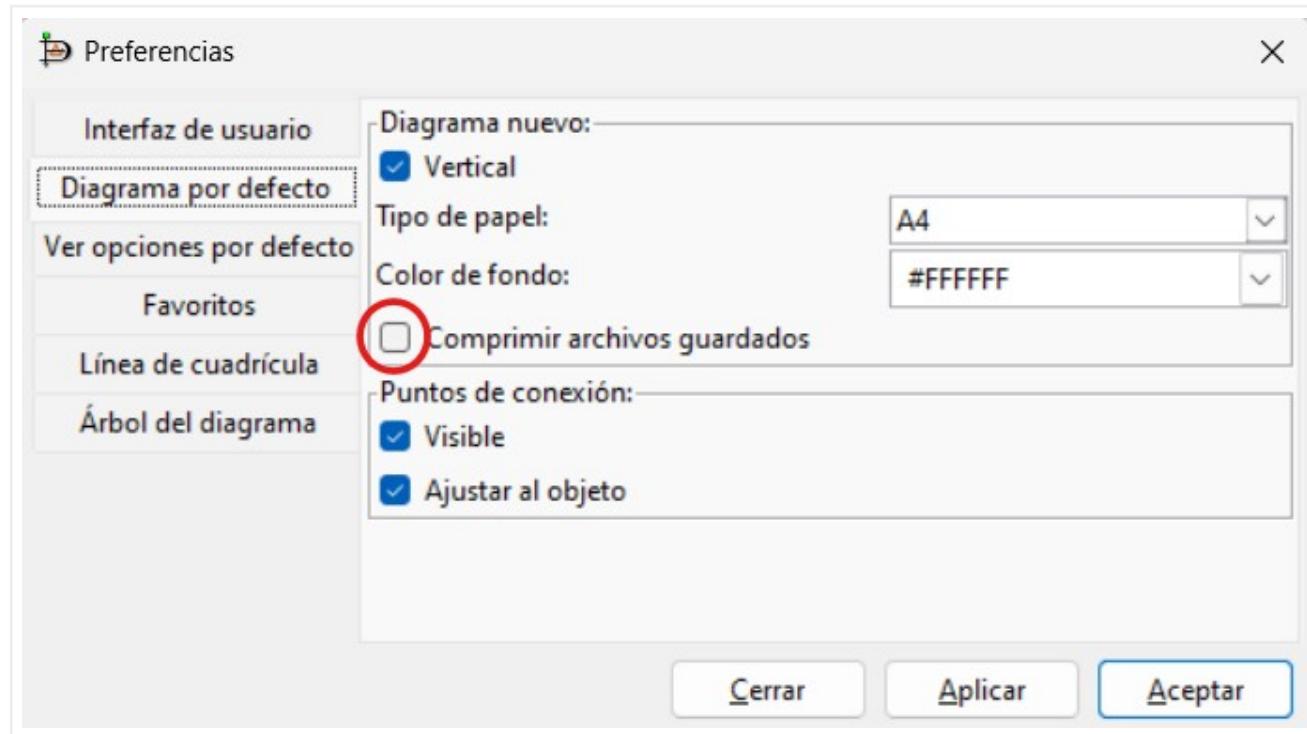
Diseñar diagramas UML puede hacerse de diversas maneras. DIA es la herramienta que usamos en clase: de código abierto, gratuita y ligera, con paletas específicas para UML, bases de datos y diagramas de flujo.



El lienzo a la derecha y las paletas de herramientas a la izquierda, con la paleta UML seleccionada

DIA: configuración importante

Para poder generar código a partir de un archivo DIA, hay que desactivar la compresión antes de guardar. Archivo → Preferencias → Diagrama por defecto, y desmarcar "Comprimir archivos guardados". Sin esto, dia2code no puede leer el archivo.



Archivo → Preferencias → Diagrama por defecto: desmarcar "Comprimir archivos guardados"

Herramientas visuales (drag & drop)

Estas aplicaciones ofrecen paletas de componentes UML pre-dibujados que se arrastran al lienzo con el ratón, sin escribir ninguna sintaxis. Es el enfoque más cómodo para diseñar en colaboración con otras personas en tiempo real.

Herramienta	Cómo se usa	Ventajas	A tener en cuenta
Draw.io	Navegador web o app de escritorio	Gratuito, se integra bien con Drive y GitHub, interfaz familiar	—
Lucidchart	Aplicación web	Excelente entorno colaborativo en tiempo real	La versión gratuita limita a un par de documentos
StarUML	Software de escritorio	Ingeniería inversa y generación de código integradas	Curva de aprendizaje algo mayor

Diagramación como código

Aquí el diagrama no se dibuja con el ratón: se escribe como texto plano y una herramienta lo convierte en imagen. Es la forma preferida en proyectos que usan Git, porque un fichero de texto admite "diffs" — algo imposible con una imagen.

Mermaid

```
```mermaid
classDiagram
 ClaseA -- ClaseB
```
```

Se renderiza donde haya intérprete; GitHub lo soporta de forma nativa. Es lo que hemos usado en varios diagramas sencillos de este tema.

PlantUML

```
@startuml
ClaseA -- ClaseB
@enduml
```

El pionero de este enfoque: sintaxis algo más detallada, pero con mucho más soporte de plugins en los IDE.

¿Cuál elegir? Para diagramas rápidos dentro de Markdown, Mermaid es más ligero. Para proyectos grandes con muchos tipos de diagrama, PlantUML tiene más opciones de personalización.

Plugins de entornos de desarrollo

Puedes integrar diagramas de clases en tus proyectos con "Extensiones" (NetBeans) o "Plugins" (IntelliJ, Eclipse, VS Code). La ventaja: el diagrama se genera a partir del código que ya tienes, sin dibujar nada a mano — es la misma idea que la ingeniería inversa que verás más adelante en el tema.

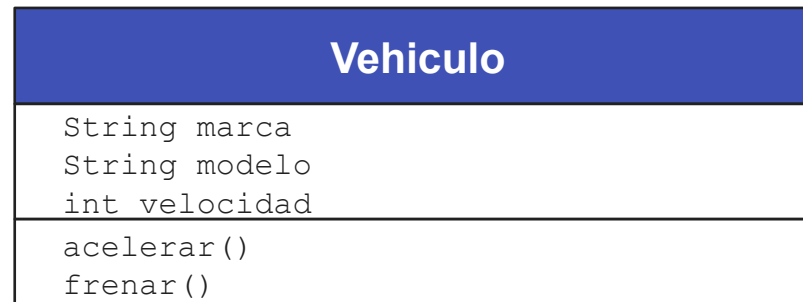
Con IntelliJ IDEA (versión Ultimate, o Community con algunos plugins) basta con "click derecho → Diagrams → Show Diagram" sobre cualquier base de código Java para que se trace solo.

¿Dibujar el diagrama o generarlo del código? Con DIA, Draw.io o Mermaid dibujas tú, para diseñar antes de escribir el código. Con un plugin del IDE, el diagrama sale después, a partir de código que ya existe: sirve para entender o documentar un proyecto ya escrito. Ambos enfoques son UML, pero resuelven momentos distintos del trabajo.

¿Qué es una clase?

Una clase es un molde o plantilla a partir de la cual se crean los objetos: agrupa datos (atributos) y comportamientos (métodos) que van a tener los objetos creados a partir de ella.

En un diagrama UML, una clase es un rectángulo de tres partes



- Nombre de la clase — arriba, en singular y con mayúscula inicial
- Atributos — en medio, las características del objeto
- Métodos (operaciones) — abajo, las acciones que puede realizar

Visibilidad (encapsulamiento)

El encapsulamiento es el principio de ocultar los detalles internos de una clase y exponer solo lo necesario. Se denota con símbolos delante de cada atributo y método.

| Símbolo | Nombre | Accesible desde... |
|---------|-----------------------|--|
| + | Público (Public) | Cualquier otra clase o parte del programa |
| - | Privado (Private) | Solo desde dentro de la misma clase (lo habitual en atributos) |
| # | Protegido (Protected) | La propia clase y sus clases hijas (herencia) |
| ~ | Paquete (Package) | Clases del mismo paquete (predeterminado en Java) |

En DIA, el desplegable de visibilidad muestra: Público, Privado, Protegido e Implementación (así llama DIA a lo que en UML es Package).

Atributos

Los atributos son las propiedades de una clase. Cada objeto creado a partir de ella tendrá sus propios valores para estos atributos.

Formato UML (empieza siempre por la visibilidad)

```
visibilidad nombre : tipo = valorPorDefecto
```

- - marca : String → atributo privado sin valor por defecto
- - saldo : double = 0 → atributo privado que empieza en 0
- + activo : boolean = true → atributo público que empieza en verdadero

Métodos (operaciones)

Los métodos determinan el comportamiento de un objeto: son las funciones definidas dentro de la clase.

```
visibilidad nombre(dirección param : tipo, ...) : tipoRetorno
```

| Dirección | Significado |
|-----------|---|
| in | Parámetro de entrada (lo más habitual, equivale a omitirlo) |
| out | Parámetro de salida (el método escribe en él) |
| inout | Entrada y salida |

- + acelerar(in incremento : int) : void
- + transferir(in cantidad : double, in destino : CuentaCorriente) : boolean

Modificador static

Un atributo o método static pertenece a la clase en sí, no a cada objeto: todos los objetos comparten el mismo valor.

```
public class Matematicas {  
    public static final double PI = 3.1416;  
  
    public static double seno(double a) {  
        return Math.sin(Math.toRadians(a));  
    }  
}  
  
// Se usa sin crear ningún objeto:  
System.out.println(Matematicas.PI);
```

En UML, los miembros estáticos se representan subrayados. En DIA se usa la convención de guiones bajos: _PI_, _seno_.

Estereotipos

Los estereotipos añaden información extra a un elemento del diagrama. Se escriben entre comillas angulares («nombre») encima o cerca del elemento.

| Estereotipo | Se aplica a | Significado |
|----------------------|-------------|--|
| «interface» | Clase | Es una interfaz |
| «abstract» | Clase | Es abstracta, no se puede instanciar |
| «utility» | Clase | Solo métodos y atributos estáticos |
| «entity» | Clase | Entidad de negocio en el modelo de datos |
| «boundary» | Clase | Interfaz de usuario o punto de interacción |
| «control» | Clase | Clase de control que gestiona la lógica |
| «enumeration» | Clase | Tipo enumerado (enum) |
| «exception» | Clase | Representa una excepción |
| «Create» / «Destroy» | Operación | Constructor / destructor de la clase |

Objetos e instanciación

Si una clase es el molde, un objeto es lo que construimos con él. Crear un objeto a partir de una clase se llama instanciación.

Todo objeto tiene tres características

- Identidad — cada objeto es único, aunque tenga los mismos datos que otro
- Estado — los valores de sus atributos en un momento dado
- Comportamiento — lo que puede hacer, definido por los métodos de su clase

```
Coche miCoche = new Coche("Toyota", "Corolla", "Rojo");  
Coche tuCoche = new Coche("Ford", "Focus", "Azul");  
// Dos objetos distintos, misma clase, estados diferentes
```

Los pilares de la POO

Clases, atributos, métodos y visibilidad son piezas sueltas. Cuatro ideas las convierten en programación orientada a objetos:

| Pilar | En una frase |
|-----------------|--|
| Encapsulamiento | Ocultar los datos internos y exponer solo lo necesario |
| Herencia | Una clase hija reutiliza atributos y métodos de una clase padre |
| Polimorfismo | El mismo método se comporta distinto según el objeto real que lo ejecuta |
| Abstracción | Definir qué hace algo sin fijar cómo lo hace |

Polimorfismo

Una misma llamada a un método puede ejecutar código distinto según el tipo real del objeto que la recibe, aunque la variable tenga un tipo más genérico.

```
interface Hablador { public void habla(); }
class Gato implements Hablador {
    public void habla() { System.out.println("Miau"); }
}
class Cuco implements Hablador {
    public void habla() { System.out.println("Cu cu"); }
}

Hablador animal1 = new Gato();
Hablador animal2 = new Cuco();
animal1.habla(); // Miau
animal2.habla(); // Cu cu
```

Java decide en tiempo de ejecución qué versión de habla() se llama, no en tiempo de compilación.

Abstracción

Abstracción es quedarte con lo esencial de un concepto e ignorar los detalles que no importan para el problema que estás resolviendo.

```
public class Empleado {  
    private double salario;    // relevante para nóminas  
    private String numeroSS;  // relevante para nóminas  
    // colorDeOjos NO aparece: es real, pero irrelevante aquí  
}
```

Las clases abstractas y las interfaces llevan la abstracción un paso más allá: declaran que un método existe (el qué) sin escribir todavía su implementación (el cómo).